

COMPUTER SCIENCE DEPARTMENT
(PROGRAMMING LANGUAGES)
TECHNICAL ASSESSMENT - FINAL PROJECT

ShopScript

A Mini Programming Language Interpreter for E-Commerce Cart and Order Simulation

Project Title	Design and Implementation of ShopScript: A Mini Programming Language for E-Commerce Simulation
Prepared by	
Section	
Date	June 19, 2026
Platform	Web-based system / browser-based interpreter

Important scope note: ShopScript is not a full production e-commerce website. It is a small educational interpreter that uses e-commerce actions as the visual execution output of a mini programming language.

1. Project Overview

ShopScript is a web-based mini programming language interpreter where users write simple code to simulate common e-commerce operations such as adding products to a cart, checking inventory, applying discounts, computing totals, creating an order, and printing a receipt. The system will run directly in a browser and will include a code editor, analyzer panel, variable table, error panel, cart output, inventory output, and receipt output.

The project shifts the original CodeCraft idea from a block-world simulation into an e-commerce simulation while preserving the required programming-language concepts. Instead of moving a character, valid code executes business actions in a store/cart environment.

Core concept: the e-commerce interface is the output of the interpreter, while the main academic requirement remains the design and implementation of a mini programming language.

2. Research-Based Direction

Internet research suggests that the project should focus on a browser code editor, a JavaScript tokenizer/parser, cart/checkout usability, and strict business-logic validation. For the final project, the recommended approach is to use reliable official documentation and research sources, while treating Reddit/community posts only as informal developer experience notes.

Area	Reliable Source Finding	How It Applies to ShopScript
Browser code editor	CodeMirror is a code editor component for the web. Monaco is also strong but heavier and not supported in mobile browsers.	Use CodeMirror for a clean web-based code editor. Monaco may be optional if the group wants a VS Code-like editor.
Parser/interpreter	Chevrotain is a JavaScript parser building toolkit for parsers, compilers, and interpreters. ANTLR is another mature parser generator that builds parse trees from grammars.	Use Chevrotain or a manual recursive-descent parser. Avoid making the parser too complex.
Learning reference	Crafting Interpreters teaches scanner, parser, semantic, and interpreter concepts through a scripting language implementation.	Use it as a conceptual guide for tokenizer, parser, AST, and interpreter flow.
E-commerce UX	NNGroup emphasizes that usable carts should show product detail, allow access to product pages, and allow easy item deletion. Baymard has long-term checkout UX research.	The simulator should clearly show cart rows, quantity, price, discount, total, and receipt logs.
Security/business logic	OWASP warns that business logic must validate price, quantity, coupons, and direct input, not only UI actions.	Semantic analysis should block negative quantity, invalid price, invalid coupon reuse, and checkout with empty cart.
Payments	PCI standards are for protecting payment data throughout the payment lifecycle. PayPal sandbox can simulate payment flows without live accounts.	Do not process real card data. Use a simulated payment status only, or sandbox mode if required later.

3. Alignment with the Project Specification

The project specification requires a mini programming language or interpreter that integrates Modules 1 to 7: language design, lexical and syntax analysis, names/scope/binding, semantic analysis, control flow, data types, and object orientation. ShopScript is aligned because every e-commerce command is processed through those compiler/interpreter stages before execution.

Specification Phase	ShopScript Implementation
Phase 1: Language Design	Define purpose, syntax style, keywords, symbols, commands, and basic program structure.
Phase 2: Lexical Analysis	Scan source code into tokens: keywords, identifiers, operators, literals, symbols, and commands.
Phase 3: Syntax Analysis	Validate grammar for declarations, expressions, cart commands, if/else, loops, and class/object syntax.
Phase 4: Names, Scope, Binding	Require variables to be declared before use; support global and block-local variables; bind identifiers to values during execution.
Phase 5: Semantic Analysis	Check types, undeclared variables, invalid operations, command parameters, quantity rules, discount rules, and checkout rules.
Phase 6: Control Flow	Support if/else, for loops, and while loops; show output logs demonstrating execution flow.
Phase 7: Data Types	Support int, float, bool, and string; handle numeric operations and int-to-float conversion.
Phase 8: OOP	Demonstrate class definition, object creation, method usage, and basic encapsulation using Product, Cart, Order, or Customer.

4. Main Goal

- Demonstrate how a mini programming language works through an e-commerce simulation.
- Show lexical analysis, syntax analysis, semantic checking, scope/binding, control flow, data types, and OOP in one unified system.
- Allow users to write simple code that produces visible cart, inventory, order, and receipt outputs.
- Keep the system educational and manageable for a final programming languages project.

5. Platform and Suggested Technologies

Component	Recommended Technology	Reason
Interface	HTML, CSS, JavaScript	Enough for a browser-based interpreter and visual output.
Code Editor	CodeMirror 6	Lightweight, extensible, web-focused editor component.
Tokenizer/Parser	JavaScript + Chevrotain or manual parser	Fits the mini-language scope and works directly in the browser.
State Storage	In-memory JavaScript objects; optional localStorage	Enough for product catalog, cart, variables, and sample programs.
Visual Output	HTML tables/cards	Better fit for e-commerce than Canvas.
Optional Styling	Dark mode, store dashboard theme	Improves presentation without expanding compiler scope.

6. System Concept

Section	Purpose
A. Code Editor	Where the user writes ShopScript commands and sample programs.
B. Analyzer Panel	Displays token list, syntax errors, semantic errors, variable table, scope table, and output logs.

Section	Purpose
C. E-Commerce Simulation Output	Displays product catalog, cart table, inventory changes, discount result, checkout status, and receipt.

Suggested Web Layout

ShopScript: E-Commerce Programming Language Interpreter				
Code Editor		Store / Cart Simulation		
		Products	Cart	Receipt
Tokens	Syntax Errors	Semantic Errors	Variables	Logs

7. Language Design

Purpose: ShopScript is designed to control an e-commerce cart and order simulation using simple programming instructions. The language is intentionally small, readable, and easy to tokenize, parse, check, and execute.

Supported Keywords

int, float, string, bool, if, else, for, while, class, new, private, public, true, false

Sample Commands

```
cart.add("Keyboard", 2);
cart.remove("Mouse");
cart.update("Monitor", 1);
inventory.check("Keyboard");
discount.apply("STUDENT10");
checkout.order();
receipt.print();
```

Supported Data Types

Data Type	Example	Use in E-Commerce
int	int qty = 2;	Quantity, stock count, loop counter.
float	float price = 999.50;	Price, discount value, cart total.
string	string product = "Keyboard";	Product name, coupon code, customer name.
bool	bool inStock = true;	Stock status, payment status, condition result.

8. Sample ShopScript Program

```
string product = "Keyboard";
int quantity = 2;
float price = 999.50;
bool inStock = true;

if (inStock == true) {
    cart.add(product, quantity);
}

if (quantity >= 2) {
    discount.apply("BULK10");
} else {
    discount.apply("NONE");
}

checkout.order();
receipt.print();
```

Expected result: the interpreter tokenizes the code, validates syntax, checks semantic rules, updates the cart, applies the discount if valid, creates a simulated order, and prints a receipt in the output panel.

9. Formal Grammar Rules

The grammar below is intentionally small so that the group can implement it within the final-project scope.

```
program      -> statement*
statement    -> declaration | assignment | command | ifStmt | forStmt | whileStmt |
classDecl | objectCreate

declaration  -> type IDENTIFIER "=" expression ";"
type         -> "int" | "float" | "string" | "bool"
assignment   -> IDENTIFIER "=" expression ";"

command      -> cartCommand | inventoryCommand | discountCommand | checkoutCommand |
receiptCommand
cartCommand  -> "cart.add" "(" value "," value ")" ";"
              | "cart.remove" "(" value ")" ";"
              | "cart.update" "(" value "," value ")" ";"
inventoryCommand -> "inventory.check" "(" value ")" ";"
discountCommand -> "discount.apply" "(" value ")" ";"
checkoutCommand -> "checkout.order" "(" ")" ";"
receiptCommand -> "receipt.print" "(" ")" ";"

ifStmt       -> "if" "(" condition ")" block ("else" block)?
forStmt      -> "for" "(" assignment condition ";" assignment ")" block
whileStmt    -> "while" "(" condition ")" block
block        -> "{" statement* "}"

classDecl    -> "class" IDENTIFIER "{" member* "}"
objectCreate -> IDENTIFIER IDENTIFIER "=" "new" IDENTIFIER "(" ")" ";"
member       -> access? type IDENTIFIER ";" | methodDecl
access       -> "private" | "public"
methodDecl   -> IDENTIFIER "(" parameterList? ")" block
```

10. Lexical Analysis Plan

The lexical analyzer scans the source code from left to right and groups characters into meaningful tokens. It ignores whitespace and comments, but keeps line and column numbers for error messages.

Token Category	Examples
Keywords	int, float, string, bool, if, else, for, while, class, new, private, public
Identifiers	product, quantity, price, inStock, item

Token Category	Examples
Operators	=, +, -, *, /, ==, !=, <, >, <=, >=, &&,
Literals	123, 999.50, "Keyboard", true, false
Symbols	;, comma, parentheses, braces, dot
Commands	cart.add, inventory.check, discount.apply, checkout.order, receipt.print

11. Syntax Analysis Plan

- Detect missing semicolons after declarations, assignments, and commands.
- Detect unmatched parentheses and braces.
- Validate the correct command format, such as `cart.add(product, quantity);`
- Validate if/else, for, and while structure.
- Validate class and object creation format.
- Create a simple parse tree or internal statement list before execution.

12. Names, Scope, and Binding

Rule	Description
Declaration before use	A variable must be declared before being used in expressions or commands.
Global scope	Variables declared outside blocks can be used anywhere after declaration.
Local scope	Variables declared inside if/else, for, while, or class methods are local to that block.
Binding	Each variable name is bound to its data type, current value, and scope level during execution.
Duplicate name check	The same variable name cannot be declared twice in the same scope.

13. Semantic Analysis Plan

Semantic analysis checks if the program makes sense after the syntax is already valid. This is where the project becomes stronger than a basic syntax checker.

Semantic Rule	Example Error
Undeclared variable checking	cart.add(product, 2); where product was never declared.
Type checking	int qty = "two"; is invalid because a string is assigned to an int.
Command parameter checking	cart.add(2, "Keyboard"); is invalid because the first parameter should be a product string.
Quantity validation	cart.add("Mouse", -1); is invalid because quantity must be positive.
Inventory validation	Checkout fails if ordered quantity is greater than available stock.
Price validation	A product price cannot be negative or changed by client command during checkout.
Discount validation	A coupon cannot be applied twice unless allowed.
Checkout validation	checkout.order(); fails if the cart is empty.
OOP validation	An object must be created before calling its method.

14. Control Flow Implementation

- if/else is used for decisions such as stock availability and discount eligibility.
- for loops are used for repeated actions, such as adding multiple products from a list.
- while loops are used for repeated checks, but the interpreter should include a loop safety limit to avoid infinite loops.
- The output log should show each command execution step so the flow is easy to demonstrate.

Control Flow Example

```
int i = 0;
while (i < 3) {
    cart.add("Pencil", 1);
    i = i + 1;
}

if (cart.total() >= 500.00) {
    discount.apply("SAVE50");
}
```

15. Data Types and Type System

Rule	Description
Integer operations	int values can be used for quantity, stock, and loop counters.
Float operations	float values can be used for price, subtotal, discount, and total.
String operations	string values can be compared and passed to product/coupon commands.
Boolean operations	bool values can be used directly in if/while conditions.
Allowed conversion	int can be converted to float during price calculations.
Blocked conversion	string cannot be automatically converted to int or float.
Invalid arithmetic	bool and string are not allowed in numeric arithmetic.

16. Object-Oriented Features

ShopScript will include a simple OOP demonstration. The OOP part does not need to be a full professional object system; it only needs to demonstrate class definition, object creation, method usage, and basic encapsulation.

```
class Product {
```

```
private string name;
private float price;
private int stock;

setProduct(string n, float p, int s) {
    name = n;
    price = p;
    stock = s;
}

Product item = new Product();
item.setProduct("Keyboard", 999.50, 10);
```

OOP Requirement	ShopScript Example
Class definition	class Product { ... }
Object creation	Product item = new Product();
Method usage	item.setProduct(...);
Encapsulation	private fields such as name, price, and stock.

17. Main Web Features

Required Feature	Description
Code input area	User writes ShopScript code.
Run button	Runs the analyzer and interpreter.
Token display	Shows token type, lexeme, line, and column.
Syntax checker	Shows grammar-related errors.
Semantic checker	Shows type, variable, cart, inventory, and discount errors.
Variable table	Shows name, type, value, and scope.
Cart output	Shows product, quantity, price, subtotal, discount, and total.
Inventory output	Shows product stock before and after execution.
Receipt output	Shows final order summary and execution logs.

Optional Aesthetic Features

- Dark mode interface.
- Store-dashboard style panels.
- Green success messages and red error messages.
- Current line highlighting during execution.
- Save/load sample programs using localStorage.
- Preset sample buttons: Basic Cart, Discount, Inventory Check, Loop Demo, OOP Demo.

18. Expected Final Output

- Accept user code from the web page.
- Analyze the code into tokens.
- Check syntax errors.
- Check semantic errors.
- Execute valid e-commerce commands.
- Show variables, cart, inventory, receipt, and output logs.
- Demonstrate variables, loops, conditions, data types, and basic OOP.

19. Recommended Group Scope

Version	Features
Submission Minimum	Code editor, tokenizer, syntax checker, semantic checker, variables, if/else, for/while, four data types, cart.add, checkout.order, receipt.print, and one simple OOP sample.
Improved Version	Product catalog, inventory checking, discount.apply, variable table, scope table, output logs, better error messages, and sample programs.
Best Version	CodeMirror editor, current-line highlighting, save/load programs, animated receipt/cart updates, multiple product types, and a debug step-through mode.

20. Development Roadmap

Step	Task	Output
1	Finalize language syntax and grammar	Keyword list, command list, grammar rules, sample code.

Step	Task	Output
2	Build tokenizer	Token table with type, value, line, and column.
3	Build parser/syntax checker	Syntax error messages and valid statement list.
4	Build symbol table	Variable name, type, value, and scope tracking.
5	Build semantic checker	Type, declaration, command, cart, inventory, and discount validation.
6	Build interpreter	Execution of valid commands and output logs.
7	Build e-commerce output	Cart, inventory, checkout status, receipt.
8	Add OOP demo	Product/Cart/Order class sample.
9	Test and polish UI	Demo-ready final project.

21. Prepared Q&A; Answers

Question	Answer
What is the purpose of your designed programming language?	The purpose of ShopScript is to simulate e-commerce cart and order operations through a simple programming language. It helps demonstrate how lexical analysis, syntax analysis, semantic checking, control flow, data types, scope, and OOP work together in an interpreter.
How does your lexical analyzer identify tokens?	It scans the input code character by character and groups text into keywords, identifiers, operators, literals, symbols, and command tokens. Each token also stores its line and column for error reporting.
What grammar rules did you define for syntax analysis?	The grammar supports declarations, assignments, expressions, commands, if/else statements, for loops, while loops, class definitions, object creation, and method calls.
How does your system handle scope and binding?	The system uses a symbol table. Global variables are stored in the global scope, while variables inside blocks or methods are local. Each identifier is bound to its type, value, and scope level.
What semantic checks are implemented in your language?	The semantic checker detects undeclared variables, wrong data types, invalid operations, invalid command parameters, negative quantities, invalid prices, duplicate coupon use, insufficient stock, and checkout with an empty cart.
What control flow structures are supported?	ShopScript supports if/else statements, for loops, and while loops. The output logs show how the interpreter follows the execution path.
What data types are supported and how are they used?	The language supports int, float, bool, and string. These are used for quantity, price, stock status, payment status, product names, coupon codes, and conditions.

22. Short Project Description

ShopScript is a web-based mini programming language interpreter that allows users to write simple code for e-commerce cart and order simulation. It accepts code from a browser editor, performs lexical analysis, syntax checking, and semantic checking, then executes valid commands such as adding products to a cart, checking inventory, applying discounts, creating an order, and printing a receipt. The project demonstrates programming language design concepts through an interactive and practical e-commerce scenario.

23. Final Recommendation

The e-commerce shift is recommended because it is easier to explain in a defense than a game-like simulation, while still satisfying the programming language final-project specification. The safest final scope is a simulated store/cart interpreter, not a real e-commerce platform with actual accounts, payments, or database transactions.

24. References and Internet Research Sources

[S1] CodeMirror	Official documentation describes CodeMirror as a web code editor component with extensible APIs.	https://codemirror.net/
[S2] Monaco Editor	Official site describes Monaco as the editor that powers VS Code and lists browser support limits.	https://microsoft.github.io/monaco-editor/
[S3] Chevrotain	Official site describes Chevrotain as a JavaScript parser building toolkit for parsers, compilers, and interpreters.	https://chevrotain.io/
[S4] ANTLR	Official site describes ANTLR as a parser generator for reading, processing, executing, or translating structured text.	https://www.antlr.org/
[S5] Crafting Interpreters	Online book for implementing a scripting language, including parsing and semantics.	https://craftinginterpreters.com/
[S6] MDN Web Storage API	MDN documents browser key/value storage and localStorage behavior.	https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
[S7] MDN JavaScript Classes	MDN explains classes as templates that encapsulate data with code.	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes
[S8] Nielsen Norman Group	NNGroup cart UX article says usable carts provide product detail, access to product pages, and easy item deletion.	https://www.nngroup.com/articles/shopping-cart/
[S9] Baymard Institute	Baymard provides long-running e-commerce cart and checkout usability research.	https://baymard.com/research/checkout-usability
[S10] OWASP WSTG	OWASP Web Security Testing Guide provides web application security testing guidance.	https://owasp.org/www-project-web-security-testing-guide/
[S11] OWASP Business Logic Security	OWASP highlights price, quantity, and coupon misuse cases that map well to semantic checking rules.	https://cheatsheetseries.owasp.org/cheatsheets/Business_Logic_Security_Cheat_Sheet.html
[S12] PCI Security Standards Council	PCI standards protect payment data throughout the payment lifecycle; real payment handling is out of scope.	https://www.pcisecuritystandards.org/standards/
[S13] PayPal Sandbox	PayPal sandbox is a virtual testing environment for simulated payments; optional only if the group expands scope.	https://developer.paypal.com/tools/sandbox/

Note on Reddit/community sources: Reddit discussions can be useful for informal developer experience, but official documentation, security standards, and research sources are stronger for the written project plan.